

Normalization

ER and Mapping => When a company does not have a system or database.

Normalization => When we already have a design and we want improve and enhance it. This design can be database or Excel Sheets. We already have tables or structures and we want to check that they are appropriate or not (minimal duplication and inconsistencies).

Normalization : The process of structuring data to minimize duplication and inconsistencies.

The process usually involves breaking down a single table into two or more tables and defining relationships between those tables.

Normalization is usually done in stages or steps, with each stage applying some rules to the type of information which can be stored in a table (1nf , 2nf , ...).

Goal of normalization :

- Avoid having different copies of the data (Inconsistency).
- Avoid unnecessarily duplication.
- Avoid Anomalies => DML anomalies (working but with a behavior that I don't want)
 - o Insertion anomalies : adding new rows forces users to create duplicate data.
 - o Deletion anomalies : deleting rows may cause a loss of data that would be needed for other future rows.
 - o Modification anomalies : changing data in a row forces changes to other rows because of duplication.
- Functional Dependency : a constraint between two attributes (columns) or two sets of columns (relationships between columns), ex: $A \Rightarrow B$ means A determines B , OR B depends on A
 - o Unique (A) => New (B) Ex: Primary Key
 - o Ex: EmpId => name , salary ,
 - o Ex (composite PK) : EmpId & ProjectNo => WorkingHours
 - o So the table is Perfect when all the columns depends in the PK and FULL FUNCTIONAL DEPENDENCY (PK is the determinant , columns as name, age, salary functional depend on the PK)
 - Full Functional Dependency : Attribute is fully Functional dependency on a PK if it's value is determined by the whole PK.
 - Partial Functional Dependency : if attribute's value is determined by a PART of the PK (Valid only with composite PK)
 - Transitive Functional Dependency : if attribute's value is determined by another non-key attribute , which is self determined by PK (ex: from the Zip code I can get the city .. the two columns are not PK and are in the same table)
 - o So having Transitive Functional Dependency or Partial Functional dependency is WRONG !!

Note : A table should not have more than one entity type.

Ex: Primary Key (composite => SID, Subject)

SID	Sname	Bdate	City	ZipCode	Subject	Grade	Teacher
1	Ahmed	1/1/1980	Cairo	1010	DB	A	Hany
1	Ahmed	1/1/1980	Cairo	1010	Math	B	Eman
1	Ahmed	1/1/1980	Cairo	1010	WinXP	A	khalid
2	Ali	1/1/1983	Alex	1111	DB	B	Hany
2	Ali	1/1/1983	Alex	1111	SWE	B	Heba
3	Mohamed	1/1/1990	Mansoura	1210	NC	C	Mona

Here anomalies exist , because we combines two themes (two entity types [Subject and Student]) into one relation. This results in duplication and unnecessary dependency between the entities (ex: if we want to add a subject then we must have a student , and vice versa)

SID	SName	Birthdate	City	Zip Code	Subject	Grade	Teacher
1	Ahmed	1/1/1980	Cairo	1010	DB	A	Hany
1	Ahmed	1/1/1980	Cairo	1010	Math	B	Eman
1	Ahmed	1/1/1980	Cairo	1010	WinXP	A	khalid
2	Ali	1/1/1983	Alex	1111	DB	B	Hany
2	Ali	1/1/1983	Alex	1111	SWE	B	Heba
3	Mohamed	1/1/1990	Cairo	1010	NC	C	Mona

Full Functional Dependency

Sid,Subject → Grade

Partial Functional Dependency

Sid → SName
Subject → Teacher

Transitive Functional Dependency

ZipCode → City

To sum up : we do normalization when we have :

- 1- Inconsistency (different copies of data)
- 2- Duplication of data
- 3- DML anomalies
- 4- Partial Functional dependency
- 5- Transitive functional dependency

Normalization steps?

- First of all , we consider the table without any changes in the (ZERO NORMAL FORM)
- Remove multivalued Attributes OR repeating groups to go to the (FIRST NORMAL FORM)
- Remove partial dependencies (see only tables with composite PK) to go to the (SECOND NORMAL FORM)
- Remove transitive dependencies to go to the (THIRD NORMAL FORM)

1nf (first normal form) : the relation is in the first normal form if it contains no multi valued or composite attributes. Done by removing repeating groups (columns that cause repeating , ex: phones , ...) to a new table with carrying the PK of the current table as a FK in the new table. All columns (fields) must be atomic (no repeating items in columns).

SID	SName	Birthdate	City	Zip Code	Subject	Grade	Teacher
1	Ahmed	1/1/1980	Cairo	1010	DB	A	Hany
1	Ahmed	1/1/1980	Cairo	1010	Math	B	Eman
1	Ahmed	1/1/1980	Cairo	1010	WinXP	A	khalid
2	Ali	1/1/1983	Alex	1111	DB	B	Hany
2	Ali	1/1/1983	Alex	1111	SWE	B	Heba
3	Mohamed	1/1/1990	Cairo	1010	NC	C	Mona

SID	SName	Birthdate	City	Zip Code	Subject	Grade	Teacher
1	Ahmed	1/1/1980	Cairo	1010	DB	A	Hany
					Math	B	Eman
					WinXP	A	khalid
2	Ali	1/1/1983	Alex	1111	DB	B	Hany
					SWE	B	Heba
3	Mohamed	1/1/1990	Cairo	1010	NC	C	Mona

Repeating Groups Or multivalued

It results into :

1NF

Student(SID, Sname, Birthdate, City, Zip Code)

SID	SName	Birthdate	City	Zip Code
1	Ahmed	1/1/1980	Cairo	1010
2	Ali	1/1/1983	Alex	1111
3	Mohamed	1/1/1990	Cairo	1010

Stud_Subject (SID, Subject, Grade, Teacher)

SID	Subject	Grade	Teacher
1	DB	A	Hany
1	Math	B	Eman
1	WinXP	A	khalid
2	DB	B	Hany
2	SWE	B	Heba
3	NC	C	Mona

2nf (second normal form) : the relation is in the second normal form if it is in first normal form AND every non-key attribute is fully functionally dependent on the PK (so remove partial functional dependencies , so no non-key attributes depends on just part of the key) , so here in this step see all the tables having composite PK

Note : in the previous example , the first table is already in the second normal form , because it has a simple PK.

2NF

Student(SID, Sname, Birthdate, City, Zip Code)

SID	SName	Birthdate	City	Zip Code
1	Ahmed	1/1/1980	Cairo	1010
2	Ali	1/1/1983	Alex	1111
3	Mohamed	1/1/1990	Cairo	1010

2NF
Because there is no
Composite PK

Stud_Subject (SID, Subject, Grade, Teacher)

SID	Subject	Grade	Teacher
1	DB	A	Hany
1	Math	B	Eman
1	WinXP	A	khalid
2	DB	B	Hany
2	SWE	B	Heba
3	NC	C	Mona

SID, Subject → Grade.....FFD
Subject → Teacher.....PFD

After transforming to the second normal form :

2NF

Student(SID, Sname, Birthdate, City, Zip Code)

SID	SName	Birthdate	City	Zip Code
1	Ahmed	1/1/1980	Cairo	1010
2	Ali	1/1/1983	Alex	1111
3	Mohamed	1/1/1990	Mansoura	1210

Stud_Subject (SID, Subject, Grade)

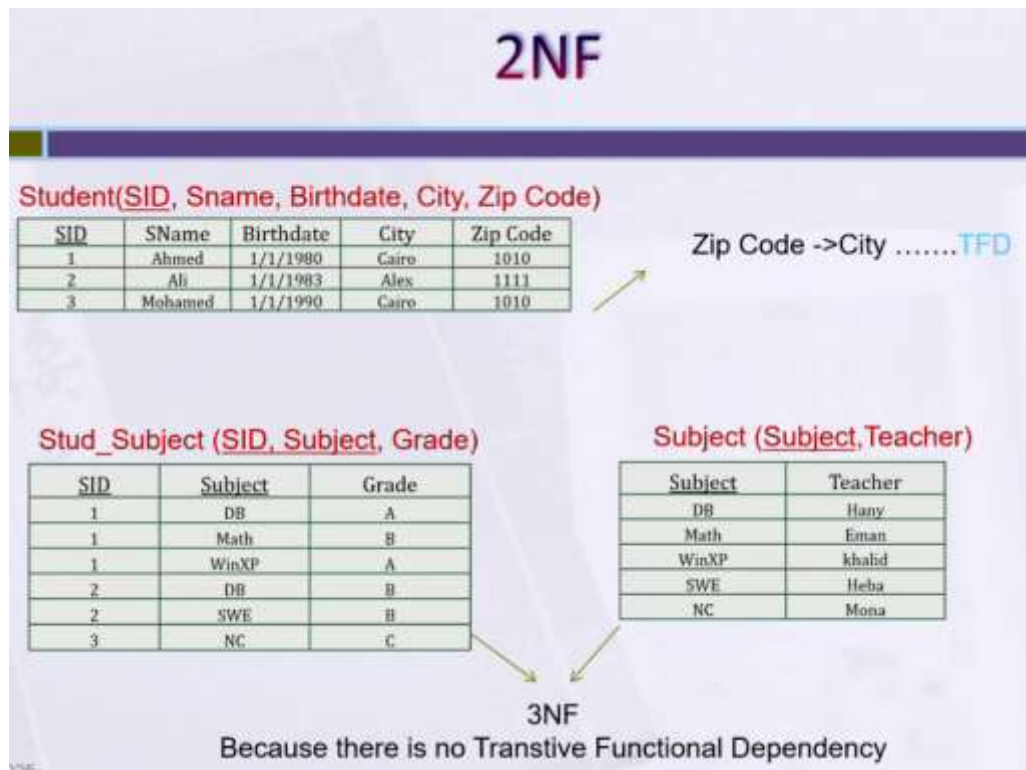
SID	Subject	Grade
1	DB	A
1	Math	B
1	WinXP	A
2	DB	B
2	SWE	B
3	NC	C

Subject (Subject, Teacher)

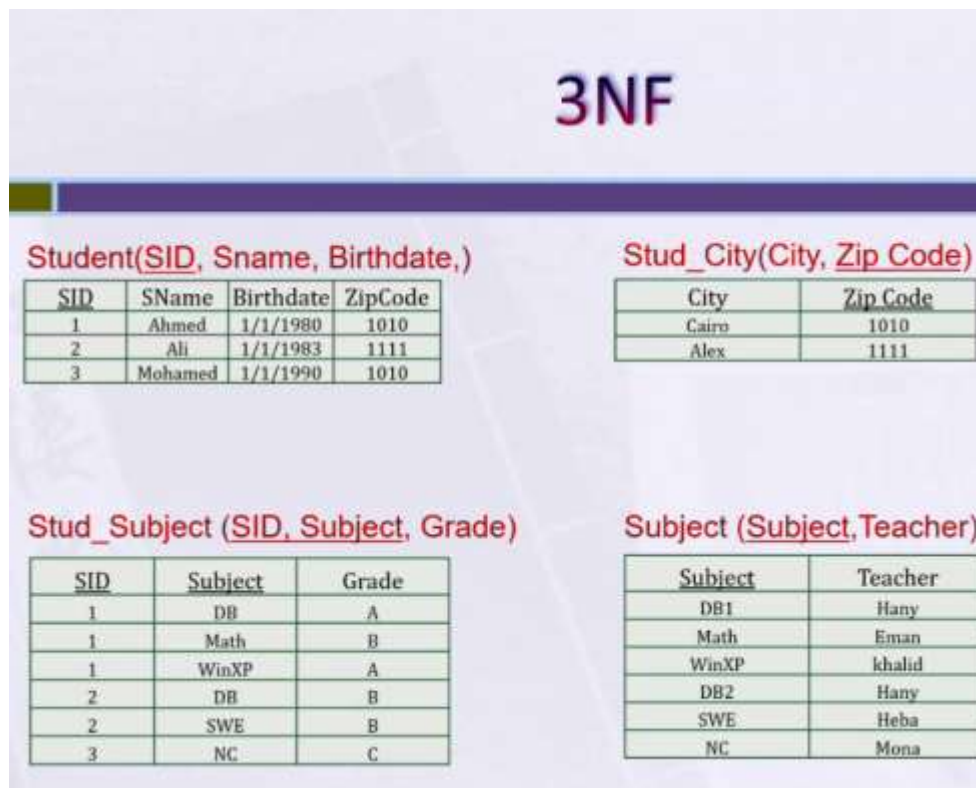
Subject	Teacher
DB	Hany
Math	Eman
WinXP	khalid
SWE	Heba
NC	Mona

3nf (third normal form) : the relation is in the third normal form if it is in second normal form AND no transitive dependencies (one attribute functionally determines a second , which functionally determines a third)

Note : in the previous example , the second and third tables are already in the third normal.



So the result is :



Note : We choose to keep the Zipcode in the first table , to be a PK and also a FK in the second table (working with numbers is better than strings)

Example 2 :

ITI Example

ITI Students Sheet

Platform Name : SWE **Platform Description:** Software Engineering
Graduate Manager: Bahaa

Appno	Name	F-code	Faculty	Address	Telno	Grade	Att. Hrs	Sdate
123	Ahmed	SC-phy	Science	Haram	3386842	A	600	14 Sep
124	Mona	Eng-cs	Engineering	Dokki	3389745, 3389744, 5123445	B	591	15 Sep
127	Ali	Com-ac	Commerce	Nasr City	2241593, 2222345	A	550	21 Sep
223	Karim	Med-bio	Medicine	Sheraton	2286845	C	600	14 Sep

1NF :

- › **Platform :** pfname , pfdesc , pfManager
- › **Students_pf:** pfname, appno, name , faculty , F-Code, address, grade, attd , start_date
- › **Std_Tel:** appno, telno

2NF

- › **Students:** appno, name , faculty , FCode, address
- › **Students_pf:** pfname, appno, grade, attd , start_date

Unchanged Tables

- › **Platform :** pfname , pfdesc , pfManager
- › **Std_Tel:** appno, telno

3NF

- › **Students:** appno, name , **FCode**, address
- › **Fac_majors:** faculty , FCode

Unchanged Tables

- › **Platform :** pfname , pfdesc , pfManager
- › **Std_Tel:** **appno**, telno
- › **Students_pf:** **pfname, appno**, grade, attd , start_date

Last example :

```
table(eid,ename ,eadd ,Pid,Pname,PLID,PLname,hours)
```

1st Normal form

Employee(Eid,Ename,eadd)

Emp_Proj(Eid,Pid,Pname,PLID,PLname,Hours)

2nd Normal form

Employee(Eid,ename,eadd)

Emp_Proj(Eid,Pid, hours)

Project(Pid,Pname,PLid,PLname)

3rd Normal form

Employee(Eid,ename,eadd)

Emp_Prj(Eid,Pid,hours)

Project(Pid,Pname,PLid)

Locations(PLid,Pname)

I

Self-study (NOT A MUST) : BCNF,4nf, 5nf
